Seba Sohayok ([redacted]) — Technical Architecture & Viva Guide

Goal-Based AI Agent with Hybrid RAG & Zero-Fact Hallucination Architecture

Course: CSE 3202 (AI Sessional) | **Domain:** Bangladesh Public Administrative Services (Passport & Driving Licence) | **Platform:** FastAPI + Gemini 2.5 Flash + NumPy + Glassmorphism UI

1. Executive Summary & Foundational Rules

Seba Sohayok is a goal-based conversational agent designed to assist Bangladeshi citizens navigating complex government services. Traditional conversational LLMs (like raw ChatGPT or Gemini) invent fees, dates, and non-existent procedures. Seba Sohayok is strictly governed by **three architectural invariants**:

Rule 1: Zero-Fact Hallucination The LLM is strictly prohibited from generating fees, deadlines, documents, or office locations. All factual data is injected verbatim from human-verified official records.	**Rule 2: Bimodal Execution** Structured action clicks bypass the LLM completely, resolving in **~5-15ms**. Natural language queries flow through intent/slot extraction with cognitive grounding.	**Rule 3: Confidence-Gated Refusal** Semantic retrieval scoring below threshold (**0.62**) triggers an explicit refusal rather than speculation, ensuring 100% reliability for public citizens.

2. End-to-End System Execution Lifecycle (How the Project Runs)

The system operates in four distinct phases from command execution to response rendering:

Phase A: Server Initialization & Tunneling (run_live.py)

- **Socket Port Cleanup:** Automatically detects and frees port 8000 (killing legacy orphan processes).
- **ASGI Boot:** Launches Uvicorn hosting app.main:app asynchronously.
- **Cloudflare Zero-Trust Tunnel:** Spawns cloudflared in parallel to publish a live public HTTPS tunnel (trycloudflare.com).
- **In-Memory Ingestion:** Loads precomputed vector archive data/embeddings.npz (3072-d NumPy matrix) and validated JSON databases into RAM for sub-millisecond retrieval.

Phase B: Dual-Path Request Routing (FastAPI)

- **Fast-Path (POST /api/action):** Action chip clicks trigger direct ID lookups (e.g. passport.new.fees) in RAM. Response time: **< 15ms**. Gemini API is NOT invoked, preserving quota and eliminating latency.
- **Cognitive Chat Path (POST /api/chat):** Free-text queries are verified by in-memory rate-limiting (20 req/min, 200 req/day per IP), filtered through regex guardrails (out-of-scope & smalltalk), and passed to the Goal Agent.

Phase C: Cognitive RAG Pipeline (Gemini 2.5 Flash + Hybrid Vector Scoring)

1. **Intent & Slot Extraction:** Gemini parses user text into JSON: {domain, subtype, field}.
2. **Clarification Loop:** If the citizen's subtype is unstated (e.g. '[redacted]'), the agent halts and prompts clarification chips.
3. **Dense Vector Embedding:** Converts user query into a 3072-d vector using gemini-embedding-001.
4. **Hybrid Retrieval:** Computes Cosine Similarity across `embeddings.npz` + Lexical Keyword Boost (Bangla/English/Banglish).
5. **Confidence Gating:** Score ≥ 0.62 → proceed to synthesis; Score < 0.62 → polite refusal card.
6. **Grounded Response Synthesis:** Gemini generates 1 introductory sentence; official facts card attached 100% verbatim.

Phase D: Claude-Style Progressive UI Rendering (web/index.html)

- Live terracotta coral pulsing dots (#D9663D) with dynamic thought steps ('■■■■■■■■■■ ■■■■■■■■■■...', '■■■■■■■■■■■■■ ■■■■■■■■■■...').
- Expandable Claude Thought Bar featuring a vertical timeline with reasoning stages and confidence metrics.
- Progressive top-to-bottom streaming: Lead text streams with blinking cursor (45-58ms/word), followed by step-by-step section reveals (260ms/step), and viewport anchoring (scrollToTopOf) preventing empty scroll voids.

3. Complete File-by-File Technical Directory (Kun File-er Ki Kaj)

Detailed technical breakdown of every architectural file in the repository:

File Path	Layer / Role	Technical Responsibilities & Guarantees
`app/main.py`	FastAPI Gateway	ASGI application entrypoint, CORS configuration, static routing (/web), session management, API endpoints (/api/chat, /api/action, /api/feedback, /api/meta, /health), and Basic Auth protected Admin router.
`app/agent.py`	Goal-Based Agent Core	The core AI reasoning brain. Manages conversation state, regex pre-guardrails (intercepts Out-of-Scope land/tax/NID queries and Smalltalk greetings), triggers clarification loops, and stitches verbatim verified facts cards.
`app/retriever.py`	Hybrid Vector Search	Performs ultra-fast vectorized NumPy matrix cosine similarity over embeddings.npz, applies multilingual keyword boosting, enforces CONF_THRESHOLD = 0.62, and fetches verified record objects.
`app/index.py`	Offline Index Builder	Batch ingestion utility. Iterates through all entries in data/*.json, generates 3072-d vectors via Gemini Embedding API, normalizes vectors ($L_2=1$), and saves data/embeddings.npz.
`app/kb.py`	Knowledge Base Schema	Defines official taxonomy (2 domains, 14 subtypes, 5 slot fields). Loads and validates JSON records into structured runtime dictionaries.
`app/llm.py`	Google GenAI Gateway	Wraps official google-genai SDK. Implements JSON schema mode for intent parsing, handles retry policies on transient API failures, and formats embedding vectors.
`app/prompts.py`	Prompt Engineering	Houses few-shot system prompts, slot extraction instructions, and strict zero-hallucination boundaries for Gemini.
`app/config.py`	Configuration Store	Loads environment variables (GEMINI_API_KEY, ADMIN_USERNAME, ADMIN_PASSWORD), ports, default model names, and confidence thresholds.
`app/limits.py`	In-Memory Rate Limiter	Sliding-window token bucket algorithm per IP address (20 req/min, 200 req/day). Protects API quotas without external Redis dependency.
`app/session.py`	Session State Store	In-memory multi-turn session tracking: active domain, resolved subtype, extracted slots, and client language (BN/EN).
`app/analytics.py`	Telemetry & Auditing	Manages SQLite database (data/analytics.db) logging queries, agent decisions, latency timestamps, and citizen feedback thumbs.
`app/texts.py`	Bilingual Text Constants	Static text dictionaries in Bangla and English for system errors, out-of-scope guidance, and UI labels.
`data/passport.json`	Curated Knowledge	Human-verified knowledge base for Bangladesh e-Passport services: regular/urgent fees, online steps, documents, delivery timelines, and office jurisdictions.
`data/licence.json`	Curated Knowledge	Human-verified knowledge base for BRTA driving licences: learner permit, smart card licence, renewal, professional/non-professional exams, and fees.
`data/embeddings.npz`	Vector Matrix Archive	Pre-calculated 3072-dimensional normalized float32 embedding matrix for all curated service entries.

`web/index.html`	Client UI Console	Zero-dependency glassmorphic web client. Features Claude-style live thinking animation, expandable reasoning summary timeline, progressive top-to-bottom streaming, dark/light theme, and bilingual support.
`web/admin.html`	Admin Dashboard	Password-protected administration console. Allows human officers to inspect analytics, review pending drafts, add new gazette records, and rebuild vector indexes.
`run_live.py / .bat`	Execution Harness	Automated launch scripts. Performs port freeing on 8000, starts Uvicorn ASGI server, launches Cloudflare Tunnel, and opens citizen browser.

4. Deep Dive into RAG (Retrieval-Augmented Generation)

Retrieval-Augmented Generation in Seba Sohayok replaces black-box probabilistic text generation with a **deterministic, verifiable information retrieval pipeline**:

4.1 Semantic Chunking Strategy

Unlike naive arbitrary 500-token chunking that fragments legal text, Seba Sohayok chunks data **semantically by service function**: (Domain, Subtype, Field). Each chunk contains: Unique ID, Title, Formatted Steps/Fees, Source URL, Verification Date, and Multilingual Keywords.

4.2 Mathematical Scoring Formula

For an incoming citizen query vector **e_q** and a knowledge base document vector **e_d**:

$$\text{FinalScore}(q, d) = \text{CosineSimilarity}(e_q, e_d) + \beta \cdot \text{KeywordBoost}(q, d)$$

where $\text{CosineSimilarity}(e_q, e_d) = (e_q \cdot e_d) / (||e_q|| \times ||e_d||)$

- **e_q:** 3072-dimensional normalized query vector (gemini-embedding-001)
- **e_d:** Pre-computed normalized entry vector in data/embeddings.npz
- **KeywordBoost(q, d):** Lexical matching score covering Bangla, English, and phonetic Banglish
- β: Heuristic boost weight (0.15) prioritizing official administrative terminology

4.3 Why Direct NumPy Matrix Scanning Over Vector Databases?

In enterprise apps with 10 million unstructured PDFs, ANN vector stores (FAISS, Chroma, Milvus) are necessary. However, for a curated, high-accuracy public service guide (~70 atomic entries), external vector databases add **cold-start packaging overhead, server dependencies, and memory footprint**. A vectorized NumPy dot product across 70 vectors runs in **0.3 milliseconds**—yielding instant, 100% mathematically exact nearest neighbor retrieval.

4.4 Confidence Gating & Anti-Hallucination Barrier

If FinalScore < 0.62, the agent triggers an explicit refusal card rather than guessing. When above threshold, Gemini is strictly constrained to authoring a 1-sentence conversational bridge, while the factual body is rendered 100% verbatim from the verified record.

5. AI Agent Formulation (Russell & Norvig PEAS Model)

Formal artificial intelligence characterization as required by university curriculum:

PEAS Dimension	System Implementation Specification
Performance Measure (P)	Zero factual hallucination rate; Precision ≥ 95% at threshold 0.62; Sub-second turn latency (<15ms action chips, <2s generative path); High citizen goal resolution rate.
Environment (E)	Bilingual citizen web client; Asynchronous HTTP REST backend; Curated government knowledge base (Passports & Driving Licences).
Actuators (A)	HTTP JSON payloads, structured answer cards with official citations, interactive clarification chips, and streaming UI tokens.
Sensors (S)	Incoming HTTP POST requests containing natural language query strings, action identifiers, session cookies, and client IP addresses.
Environment Type	**Partially Observable** (user context hidden until parsed), **Sequential** (multi-turn slot filling), **Discrete** (explicit service categories), and **Deterministic** within retrieval turn.

6. Viva & Defense Guide — How to Present to Course Teacher (Sir)

Use this structured script and answers during your project viva/presentation. Delivering these exact technical arguments demonstrates deep academic and practical engineering mastery:

6.1 The 3-Minute Elevator Pitch for Sir

"Sir, existing government chatbots fail in one of two ways: Rule-based chatbots cannot understand natural human questions, while generic generative AI (like raw ChatGPT) hallucinates wrong government fees, outdated documents, and non-existent procedures.

*Our project, **Seba Sohayok**, solves this by designing a **Goal-Based AI Agent** with a **Hybrid RAG (Retrieval-Augmented Generation) Architecture**. We guarantee **Zero Fact Hallucination** because the LLM is restricted to intent extraction and conversational phrasing, while 100% of factual fees, documents, and rules are injected verbatim from human-verified government records. We also implement a **Bimodal Execution Pipeline** where structured action chips bypass the LLM completely for instant 10ms responses, saving API quota and eliminating latency."*

6.2 Top 7 Viva Questions Sir Will Ask & Exact High-Scoring Answers

Q1: Why didn't you just use standard ChatGPT or raw Gemini API without RAG?

Raw foundation LLMs suffer from factual hallucination and knowledge cutoff. In administrative public services, providing an incorrect fee (e.g. 3,000 BDT instead of 4,025 BDT) or an outdated document list causes harassment for citizens. RAG grounds every response in human-verified government gazette entries with traceable source links.

Q2: Why use a NumPy vector matrix instead of ChromaDB, FAISS, or Pinecone?

For a specialized administrative domain with ~70 atomic, high-quality entries, setting up an external vector database server introduces cold-start network latency, disk bloat, and deployment dependencies. A vectorized NumPy cosine dot-product runs in **0.3 milliseconds** directly in RAM with zero extra dependencies and 100% mathematical precision.

Q3: How does your system handle Banglish or mixed-language queries (e.g., 'urgent passport fee koto')?

First, the Gemini-embedding-001 model is trained on multilingual and transliterated corpora. Second, our Hybrid Retrieval algorithm adds a Lexical Keyword Boost that checks multilingual keyword lists (Bangla, English, and phonetic Banglish) embedded in each record.

Q4: What makes this a 'Goal-Based Agent' rather than a simple reflex chatbot?

A simple reflex chatbot responds to immediate inputs with fixed rules without memory. Seba Sohayok maintains an internal session state (tracking domain, subtype, and slot fields). When a citizen asks an ambiguous question ('[redacted]'), the agent identifies the missing goal parameter and enters an interactive clarification loop until the goal is fully resolved.

Q5: How do you prevent citizens or malicious users from exhausting your Gemini API quota?

We built an in-memory sliding-window token bucket rate limiter in app/limits.py that tracks requests per client IP (20 per minute, 200 per day), with live visual quota indicators on the client console. Furthermore, clicking action chips bypasses the LLM completely.

Q6: What is the Confidence Threshold and why is it set to 0.62?

The confidence threshold (CONF_THRESHOLD = 0.62) acts as a strict verification firewall. If the top semantic match scores below 0.62, the query is deemed out-of-scope or vague, and the agent explicitly refuses rather than guessing an unverified answer.

Q7: What modern UI/UX engineering did you implement on the client side?

We engineered a Claude-style progressive presentation: terracotta pulsing dots with dynamic reasoning steps, an expandable thought timeline bar, progressive word streaming with cursor tracking, step-by-step section reveals, and automated viewport anchoring (scrollToTopOf) preventing blank scroll voids.